

1 - Principios de diseño y elección de gráficos

En esta unidad se introducen los principios fundamentales del diseño de visualizaciones de datos, incluyendo reglas de encuadre, jerarquía visual y uso de anotaciones para maximizar la claridad del mensaje. Además, se abordan consideraciones éticas y de percepción visual para evitar sesgos y distorsiones, garantizando una presentación honesta y accesible de los datos. Al finalizar, el estudiante podrá seleccionar tipos de gráficos adecuados según el mensaje que desea comunicar y reconocer malas prácticas comunes en visualización.

¿Alguna vez has visto un gráfico que te confundió más que ayudarte a entender los datos? La visualización de datos no es solo cuestión de hacer gráficos bonitos, sino de diseñar imágenes que comuniquen información de forma clara, precisa y ética. En esta unidad, exploraremos los principios fundamentales que guían el diseño efectivo de visualizaciones, desde cómo organizar visualmente la información hasta cómo elegir el tipo de gráfico que mejor transmite el mensaje deseado. También abordaremos la importancia de la ética en la visualización, para evitar distorsiones y sesgos que puedan engañar o confundir a la audiencia. Al dominar estos conceptos, estarás mejor preparado para crear gráficos que no solo sean atractivos, sino también confiables y útiles en contextos profesionales de ciencia de datos.

Principios de diseño de visualizaciones

El diseño de visualizaciones se basa en un conjunto de reglas y heurísticas que buscan maximizar la claridad y efectividad del mensaje. Según C. Ware en *Information Visualization: Perception for Design*, la percepción visual humana es clave para entender cómo organizar y presentar datos.

Reglas de encuadre y jerarquía visual

- **Encuadre (framing):** Consiste en definir claramente el espacio visual donde se presenta la información, evitando elementos distractores y enfocando la atención en lo relevante.
- **Jerarquía visual:** Se refiere a la organización de elementos gráficos para guiar la mirada del observador, usando tamaño, color, contraste y posición para destacar lo más importante.

Ejemplo: En un gráfico de barras, la barra con el dato más relevante puede resaltarse con un color más intenso para atraer la atención.

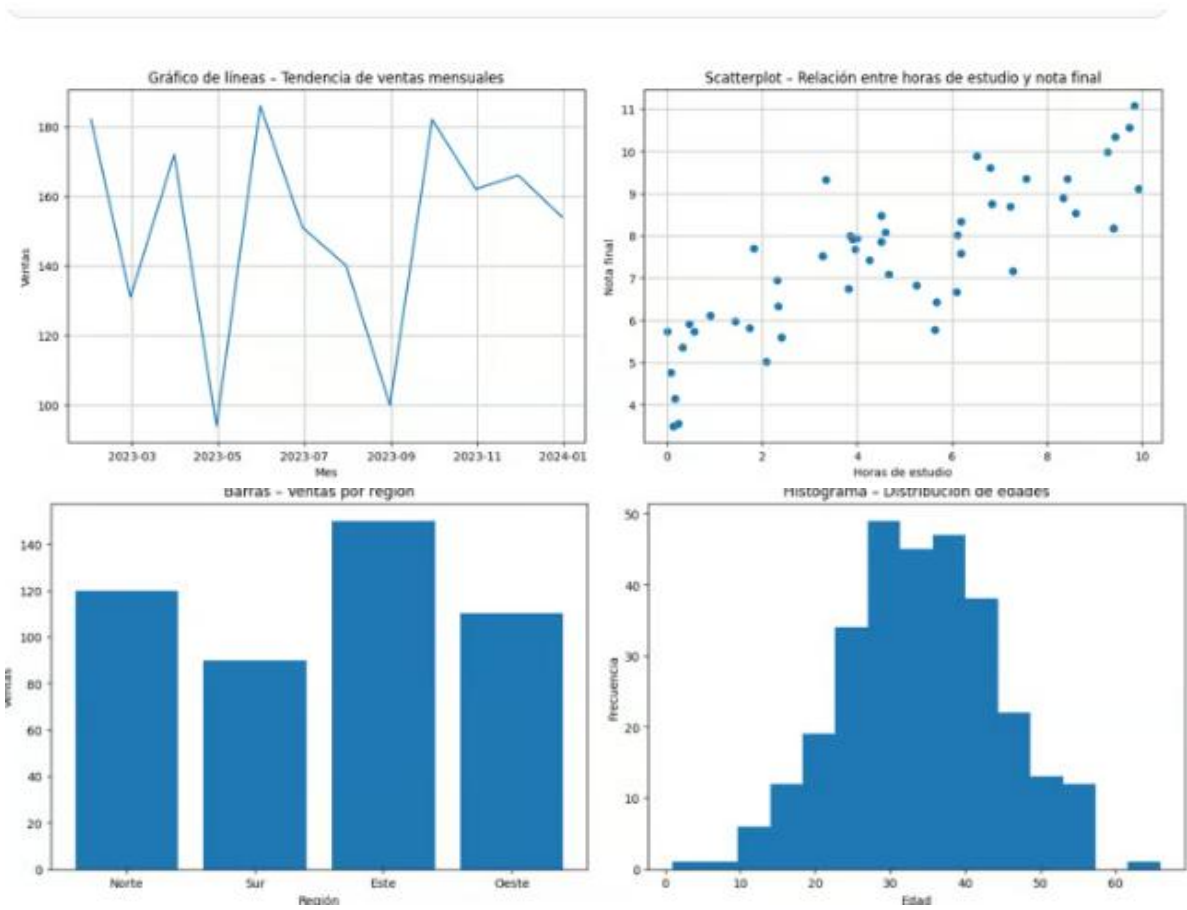
Uso de anotaciones

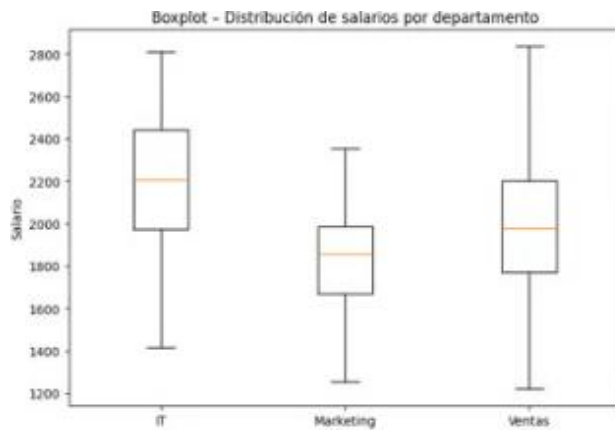
Las anotaciones (etiquetas, líneas, flechas) ayudan a clarificar puntos clave y evitar ambigüedades. Deben ser claras, concisas y no saturar el gráfico.

Elección de tipos de gráficos

Seleccionar el tipo de gráfico adecuado depende del mensaje y la naturaleza de los datos:

Tipo de gráfico	Uso recomendado
Barras	Comparación de categorías
Líneas	Series de tiempo
Boxplots	Distribución y resumen estadístico
Scatter plots	Relación entre variables





Ética y percepción visual

Sesgos y distorsiones

- Manipular ejes o escalas puede alterar la interpretación.
- Omitir datos relevantes o usar colores engañosos afecta la honestidad.

Este es un ejemplo de un gráfico deshonesto y manipulado:

Accesibilidad

- Considerar daltonismo y otras discapacidades visuales usando paletas de colores adecuadas.

Nota: Usar paletas como `colorblind-friendly` en Seaborn mejora la accesibilidad.

Presentación honesta

La visualización debe reflejar fielmente los datos, evitando exageraciones o minimizaciones que puedan inducir a error.

El mismo gráfico anterior pero representado correctamente:

Conocimiento clave: Un buen diseño visual no solo facilita la comprensión, sino que también construye confianza en la información presentada.

Los datos de ambos gráficos son simulados.

Te pasamos el código, si quieres probarlo e interactuar con Plotly:

```
import plotly.express as px
import pandas as pd
import numpy as np # Importamos numpy por si lo necesitamos, aunque no es estrictamente necesario para este código
```

```
# 1. Definición de los Datos# Creamos un diccionario con los datos de las
compañías y sus cuotasdata = {    'Compañía': ['Apple', 'Google', 'Microsoft',
'Otras Compañías'],    'Cuota de Mercado (%)': [35, 15, 10, 40]}
# Creamos un DataFrame de Pandasdf = pd.DataFrame(data)
# 2. Creación del Gráfico de Torta con Plotly Expressfig = px.pie(    df,
values='Cuota de Mercado (%)',    names='Compañía',    title='Cuota de Mercado
Global de Tecnología (35% Apple)',    # Personalización atractiva:    hole=0.4, #
Crea un gráfico de "Donut" para un look más moderno
color_discrete_sequence=px.colors.qualitative.Pastel # Usa una paleta de colores
suave)
# 3. Personalización de las Etiquetas y el Diseñofig.update_traces(    # Muestra
el porcentaje y el nombre de la compañía en la etiqueta (texto dentro de la torta)
textinfo='percent+label',    # Personalización del texto hover (lo que aparece al
pasar el ratón)    hoverinfo='label+percent',    # Aumenta el tamaño de la fuente
para las etiquetas dentro de la torta    textfont_size=14,    # Separa ligeramente
el sector de Apple para destacarlo    pull=[0.05, 0, 0, 0] )
# 4. Ajustes del Layout (Opcional, pero mejora la presentación)fig.update_layout(
uniformtext_minsize=12,    uniformtext_mode='hide',    # Ajusta el tamaño de la
leyenda    legend_title_text='Compañía')
# 5. Mostrar el Gráfico# En Colab, fig.show() muestra el gráfico
interactivofig.show()
```

En la práctica profesional de ciencia de datos, diseñar visualizaciones efectivas es crucial para comunicar hallazgos a equipos multidisciplinarios, clientes o stakeholders. Por ejemplo, un analista que presenta un informe de ventas debe elegir gráficos que resalten tendencias relevantes sin inducir a interpretaciones erróneas. Además, la ética en la visualización es fundamental para mantener la integridad del análisis y evitar decisiones basadas en información distorsionada.

Este conocimiento prepara al profesional para enfrentar desafíos reales, como seleccionar el gráfico correcto para un dashboard ejecutivo o preparar reportes que cumplan con estándares de accesibilidad y transparencia. En unidades posteriores, aplicaremos estos principios para crear composiciones avanzadas con Matplotlib y Seaborn, integrando estilos y paletas que refuercen el mensaje y la ética visual.

2 - Series temporales: trazado y transformación

Ejercicio para aplicar técnicas de visualización y transformación de series temporales usando resampling, ventanas móviles y anotaciones de eventos.

Ejercicio: Análisis y visualización de series temporales

En este ejercicio, recibirás datos de dos series temporales con valores diarios. Deberás:

1. Realizar un resampling semanal (media semanal) para ambas series.

2. Calcular la media móvil de 3 semanas para cada serie.
 3. Detectar y anotar eventos importantes (fechas específicas) en la serie 1.
 4. Imprimir los valores transformados para ambas series en formato específico para su posterior visualización.
-

Título: Series temporales: trazado y transformación (aplicado a GOOGL)

Descripción breve:

Vas a trabajar con datos diarios de la acción GOOGL. El archivo `GOOGL.csv` tiene, entre otras columnas: `Date`, `Open`, `High`, `Low`, `Close`, `Adj Close`, `Volume`. Para esta práctica usaremos **dos series**:

- **Serie A:** `Close` (precio de cierre diario)
- **Serie B:** `Volume` (volumen negociado diario; se maneja como numérico)

Objetivos (lo que debes hacer):

Te pasamos el enlace para que puedas descargar el dataset para la práctica:

<https://www.kaggle.com/datasets/varpit94/google-stock-data>

1. Cargar el CSV con fechas e indexarlo por fecha (`Date` → `datetime`, `index`).
2. Hacer un **resampling semanal** (media semanal) para ambas series.
 - La semana debe iniciar el **lunes** (es decir, usar semanas que empiecen lunes).
 - La fecha que se imprimirá para cada semana será la **fecha de inicio de la semana** (lunes).
3. Calcular la **media móvil de 3 semanas** (rolling window sobre la serie ya resampleada semanalmente) para cada serie.
 - Usar `window=3` y **solo** mostrar los valores cuando la ventana está completa (comportamiento estándar: `min_periods=3`).
4. **Detectar y anotar eventos:** el/la docente o vos podés suministrar una lista de eventos (fecha + etiqueta). El código marcará en los gráficos los eventos que coincidan exactamente con fechas presentes en la serie original.
 - Puedes pasar los eventos subiendo un archivo `events.csv` con columnas `Date`, `Label`, o editar en la celda la lista `events_manual`.
5. **Imprimir** por stdout los resultados en este orden EXACTO y con estos encabezados:
 - `Resampled Weekly A:` seguido de líneas `YYYY-MM-DD <valor>` (fecha inicio de semana y media)
 - `Resampled Weekly B:` igual para B

- Rolling 3-Week A:
 - Rolling 3-Week B:
 - Eventos: listar solo los eventos cuyas fechas aparecen en el índice original (formato YYYY-MM-DD <Etiqueta>)
6. Generar gráficos (mostrar en pantalla, **no guardar**):
 - Panel con serie diaria (A y B en ejes separados), con eventos anotados.
 - Serie semanal (media) y su rolling 3-week superpuestas y anotadas.
 - Gráfico de barras del volumen semanal (opcional: con color y anotaciones).
 7. Comentar brevemente los resultados observados (puntos altos/bajos, efecto del smoothing, eventos marcados).

Formato de entrega:

- Notebook ejecutable que genere en pantalla los gráficos y las impresiones en consola.

Este ejercicio permite aplicar técnicas de resampling, rolling windows y anotaciones en series temporales, fundamentales para análisis avanzado y visualización especializada.

3 - Preparación de datos con Pandas y NumPy

Ejercicio práctico para introducir el uso de Pandas y NumPy en la preparación y transformación de datos para visualización. El alumno deberá limpiar, agregar y transformar datos numéricos para obtener resultados listos para graficar.

Introducción

En ciencia de datos, escribir código que *funcione* no es suficiente.

Un análisis profesional debe ser **reproducible**, es decir, cualquier persona debe poder ejecutar el notebook desde cero y obtener **los mismos resultados**, sin depender del orden de ejecución, del entorno o de estados previos.

En esta práctica trabajaremos sobre un escenario realista de negocio (e-commerce) y aplicaremos **buenas prácticas de programación en notebooks**, combinando:

- generación controlada de datos,

- funciones bien definidas,
- validación mediante tests,
- y ejecución ordenada del flujo completo.

El objetivo no es solo resolver un problema lógico, sino **aprender a estructurar notebooks como lo haría un/a data scientist en un entorno profesional.**

Objetivos de la práctica

Al finalizar el ejercicio deberías ser capaz de:

- Comprender qué significa reproducibilidad en análisis de datos.
- Controlar resultados aleatorios mediante semillas.
- Organizar el código en funciones claras y reutilizables.
- Implementar un algoritmo simple de forma eficiente.
- Ejecutar todo el flujo desde una única función principal (`main()`).
- Validar resultados mediante tests locales.
- Ejecutar el notebook completo sin errores ni dependencias ocultas.

Contexto de negocio

Trabajaremos con un **dataset simulado de e-commerce**, que representa información típica de una tienda online:

- fecha de compra,
- segmento de cliente,
- región,
- categoría de producto,
- precio,

- cantidad,
- descuento,
- ventas netas.

Aunque los datos sean simulados, el objetivo es que **tengan estructura realista**, similar a la que encontrarías en un entorno laboral.

Este dataset será utilizado luego para **aplicar la lógica del problema principal** sobre una columna real del negocio.

¿Qué significa que un notebook sea reproducible?

Un notebook reproducible cumple con estas condiciones:

✓ Siempre produce el mismo resultado

Si ejecutamos el notebook hoy o mañana, en otra computadora o en Google Colab, los valores obtenidos deben ser los mismos.

Para lograrlo:

- se fijan semillas aleatorias,
- no se usan valores generados “al azar” sin control,
- no se depende del orden de ejecución manual.

✓ Puede ejecutarse de arriba hacia abajo

Un notebook reproducible debe funcionar correctamente si alguien presiona:

Runtime* → *Run all

Esto implica:

- no depender de celdas previas ejecutadas “a mano”,
- no usar variables definidas fuera del flujo,
- encapsular la lógica en funciones.

✓ No depende del usuario

Por eso, en esta práctica:

- **no se usa** `input()`
- **no se lee desde `stdin`**
- los datos de ejemplo se construyen directamente en el código

Esto permite que el notebook se ejecute de forma automática, sin intervención humana.

☐ Estructura general del ejercicio

La práctica está dividida en bloques claramente definidos.

Cada bloque cumple un propósito específico.

1 Control de reproducibilidad

En este bloque se fijan las semillas aleatorias.

¿Por qué es importante?

Funciones como:

- `np.random`
- `random`
- generación de fechas
- generación de precios o cantidades

producen resultados distintos en cada ejecución si no se controlan.

Al definir una semilla fija:

```
np.random.seed(42)random.seed(42)
```

garantizamos que:

- los mismos números se generen siempre,

- el dataset sea idéntico en cada ejecución,
- los resultados puedan compararse y evaluarse.

Esto es fundamental en notebooks evaluables y en proyectos reales.

2 Función central: lógica del problema

El problema algorítmico que se trabaja es:

Calcular la suma de los números que son múltiplos de 3 o de 5.

Este problema se implementa como una **función pura**, lo que significa que:

- recibe datos,
- produce un resultado,
- no modifica variables externas,
- no depende del entorno.

La función debe:

- recorrer la colección una sola vez (eficiencia),
- convertir los valores a enteros,
- verificar la condición matemática,
- acumular el resultado.

Este enfoque es escalable incluso para listas grandes (hasta 100.000 elementos).

3 Generación del dataset simulado

En lugar de usar un archivo externo, se construye un dataset desde cero.

¿Por qué?

Porque un notebook reproducible debe ser **autocontenido**.

Dentro de esta función se deben:

- fijar semillas locales,
- generar fechas dentro del último año,
- definir variables categóricas,
- generar precios coherentes según categoría,
- calcular ventas netas a partir de reglas de negocio.

Todo el dataset se construye con pandas y se retorna como un `DataFrame`.

Esto simula una situación real donde no siempre se dispone de datos reales, pero se necesita probar lógica, modelos o reportes.

4 Tests locales

Antes de confiar en un algoritmo, debemos validarlo.

Por eso se incluyen **tests simples**, que permiten verificar que la función:

- funciona con listas vacías,
- funciona cuando no hay múltiplos,
- funciona con valores conocidos,
- produce el resultado esperado para el caso clásico 1..10.

Estos tests cumplen un rol pedagógico clave:

- enseñan a validar código,
- previenen errores silenciosos,
- refuerzan buenas prácticas profesionales.

No se utilizan librerías externas de testing para mantener el notebook simple y autocontenido.

5 Función main()

La función `main()` actúa como el **orquestador del notebook**.

Dentro de ella se realiza:

1. impresión de versiones de librerías (control de dependencias),
2. generación del dataset,
3. exploración inicial,
4. resolución del problema con ejemplos,
5. aplicación del algoritmo a datos reales,
6. visualización simple,
7. ejecución de tests.

Este patrón es muy común en proyectos reales y permite que el código:

- sea ordenado,
- sea legible,
- pueda migrarse luego a scripts o pipelines.

6 Ejecución controlada

El bloque final:

```
if __name__ == "__main__":    main()
```

garantiza que:

- el código se ejecute correctamente en notebooks,
- también funcione si se exporta como script,
- no se ejecute automáticamente al importar funciones.

Es una práctica estándar en Python profesional.

Nota: Este ejercicio simula la importancia de mantener notebooks limpios y reproducibles, aplicando un problema sencillo pero con énfasis en buenas prácticas de código y documentación.

****¡**Manos a la obra! Implementa la función que realice esta preparación de datos usando Pandas y NumPy.

4 - Análisis multivariado y visualizaciones multidimensionales

Introducción a técnicas de visualización para análisis multivariado, incluyendo pairplots, scatter matrix y heatmaps de correlación, con énfasis en la interpretación y reducción de complejidad visual para comunicar relaciones entre múltiples variables.

¿Alguna vez te has enfrentado a un conjunto de datos con múltiples variables y te has preguntado cómo visualizar las relaciones entre ellas de forma clara y efectiva? En el análisis de datos, entender cómo interactúan varias variables simultáneamente es clave para descubrir patrones, correlaciones y posibles insights que no se aprecian en análisis univariados o bivariados.

En esta unidad, exploraremos técnicas fundamentales para el análisis multivariado mediante visualizaciones multidimensionales. Aprenderás a utilizar herramientas como los *pairplots*, *scatter matrix* y *heatmaps* de correlación, que te permitirán representar y analizar múltiples variables a la vez. Además, abordaremos estrategias para reducir la complejidad visual, facilitando la interpretación y comunicación de resultados.

Este conocimiento complementa lo visto en unidades anteriores sobre series temporales y distribuciones, y te preparará para integrar visualizaciones interactivas y exportables en reportes profesionales, que veremos en la siguiente unidad.

Análisis multivariado y gráficos multidimensionales

El *análisis multivariado* se refiere al estudio simultáneo de más de dos variables para entender sus relaciones y patrones conjuntos. En visualización, esto implica representar múltiples dimensiones de datos en gráficos que faciliten la interpretación.

Pairplots y Scatter Matrix

- **Pairplot:** Es una matriz de gráficos que muestra la relación entre cada par de variables numéricas en un dataset. Cada celda contiene un gráfico de dispersión (*scatter plot*) para un par de variables, y en la diagonal se suelen mostrar histogramas o densidades para cada variable individual.
- **Scatter Matrix:** Similar al pairplot, es una matriz de gráficos de dispersión que permite observar visualmente correlaciones y distribuciones conjuntas.

Ejemplo en la industria: En análisis financiero, un pairplot puede mostrar cómo se relacionan diferentes indicadores económicos (PIB, inflación, tasa de interés) para detectar patrones o anomalías.

Heatmaps de Correlación

Un *heatmap* de correlación es una representación gráfica de la matriz de correlaciones entre variables, donde los valores se codifican con colores. Permite identificar rápidamente qué variables están positivamente o negativamente correlacionadas.

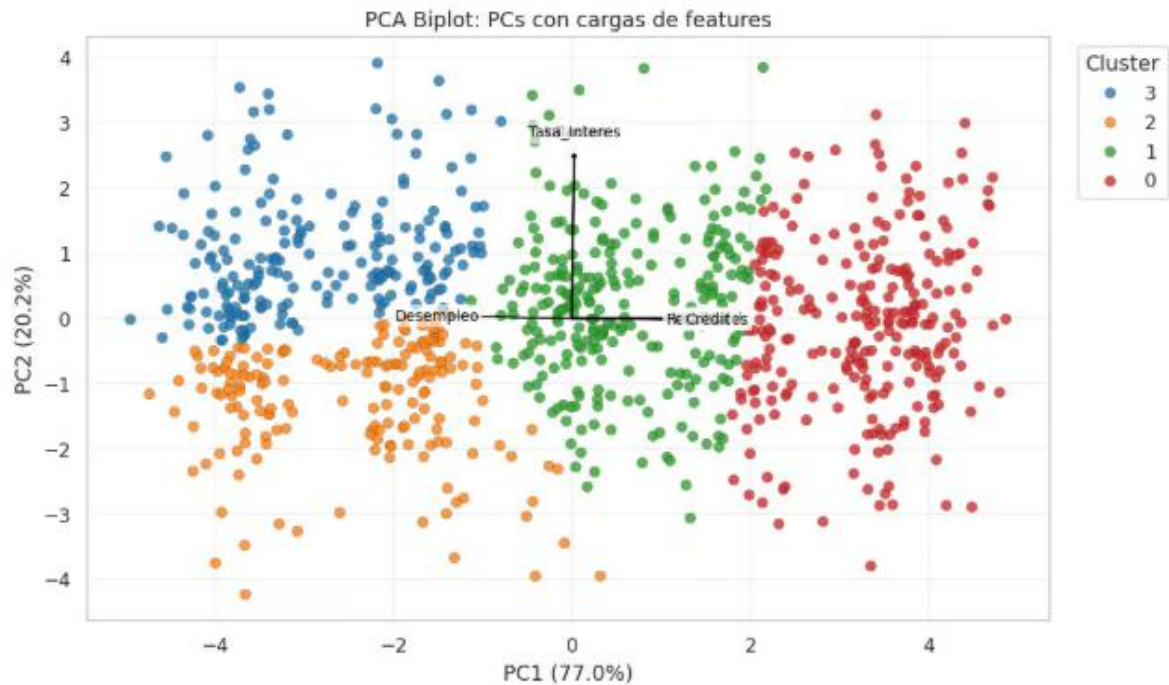
- La correlación se mide comúnmente con el coeficiente de Pearson, que varía entre -1 y 1.
- En el heatmap, colores cálidos (rojos) pueden indicar correlaciones positivas fuertes, y colores fríos (azules) correlaciones negativas.

Ejemplo en ciencia de datos: Un heatmap ayuda a seleccionar variables relevantes para modelos predictivos, evitando multicolinealidad.

Técnicas para Reducir la Complejidad Visual

Visualizar muchas variables simultáneamente puede generar gráficos saturados y difíciles de interpretar. Algunas estrategias para mitigar esto son:

- **Selección de variables clave:** Elegir solo las variables más relevantes para el análisis.
- **Agrupación o segmentación:** Dividir el dataset en subgrupos para análisis más focalizados.
- **Uso de paletas de colores y estilos consistentes:** Facilita la distinción visual sin sobrecargar.
- **Interactividad (vista previa):** Aunque no se cubre en detalle aquí, preparar visualizaciones para ser interactivas ayuda a explorar dimensiones sin saturar la vista.



Nota: En esta unidad introducimos estos conceptos para que puedas aplicarlos en tus análisis y preparar visualizaciones claras y efectivas.

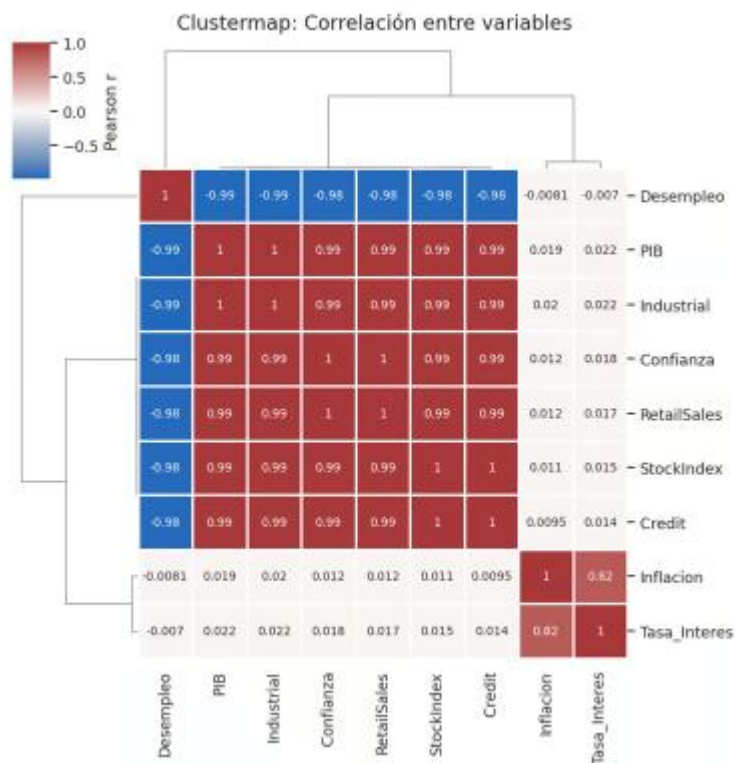
En la práctica profesional, enfrentarse a datasets con múltiples variables es común, desde análisis de clientes en marketing hasta monitoreo de sensores en IoT. Las visualizaciones multidimensionales permiten detectar relaciones complejas que no se aprecian en gráficos simples.

Por ejemplo, un científico de datos en salud pública puede usar un pairplot para explorar cómo variables como edad, presión arterial, colesterol y actividad física se relacionan entre sí y con la incidencia de enfermedades.

El heatmap de correlación es una herramienta rápida para identificar variables redundantes o altamente correlacionadas, lo que es crucial para construir modelos predictivos robustos.

Además, aplicar técnicas para reducir la complejidad visual asegura que los insights sean accesibles para audiencias técnicas y no técnicas, facilitando la toma de decisiones basada en datos.

Esta unidad te prepara para integrar estas visualizaciones en notebooks reproducibles, manteniendo claridad y profesionalismo en tus reportes y presentaciones.



5 - Estilos, paletas y accesibilidad cromática

En esta unidad, exploraremos cómo configurar estilos visuales y seleccionar paletas de colores adecuadas en Matplotlib y Seaborn, con especial énfasis en la accesibilidad cromática para asegurar que nuestras visualizaciones sean claras y efectivas para todos los públicos, incluyendo personas con daltonismo y para impresión en escala de grises.

¿Alguna vez te has preguntado por qué algunos gráficos son más fáciles de interpretar que otros? La elección adecuada de estilos y paletas de colores no solo mejora la estética, sino que también potencia la claridad y la comunicación efectiva de los datos. En esta unidad, aprenderás a configurar estilos visuales en Matplotlib y Seaborn y a seleccionar paletas de colores que se adapten a la naturaleza de tus variables, garantizando además que tus visualizaciones sean accesibles para personas con daltonismo y para impresión en escala de grises. Este conocimiento es fundamental para diseñar gráficos profesionales y éticos que comuniquen insights de forma clara y fiel a los datos, un paso clave en tu camino hacia la maestría en visualización avanzada de datos.

Estilos y configuración con rcParams

Matplotlib permite personalizar la apariencia de tus gráficos mediante el objeto `rcParams`, un diccionario que controla parámetros globales como colores, tamaños de fuente, estilos de línea y más. Por ejemplo, puedes cambiar el estilo predeterminado con:

python

```
import matplotlib.pyplot as pltplt.style.use('seaborn-darkgrid')
```

Seaborn, construido sobre Matplotlib, ofrece estilos predefinidos que facilitan la creación de gráficos estéticamente agradables y consistentes, como 'darkgrid', 'whitegrid', 'ticks', entre otros.

Paletas de color en Matplotlib y Seaborn

Las paletas de color son conjuntos de colores usados para representar datos. Se clasifican principalmente en:

- **Cualitativas:** para variables categóricas sin orden inherente (p.ej., tipos de frutas).
- **Secuenciales:** para variables numéricas ordenadas, donde el color indica magnitud (p.ej., temperatura).
- **Divergentes:** para variables numéricas con un punto medio crítico, mostrando desviaciones en dos direcciones (p.ej., diferencias respecto a un valor base).

Seaborn facilita la selección de paletas con funciones como `color_palette()` y `cubehelix_palette()`, y ofrece paletas predefinidas accesibles y profesionales.

Accesibilidad cromática

Es crucial que las visualizaciones sean interpretables por personas con deficiencias visuales, especialmente daltonismo, y que mantengan su legibilidad al imprimirse en escala de grises.

- **Daltonismo:** afecta aproximadamente al 8% de los hombres y 0.5% de las mujeres. Las paletas deben evitar combinaciones problemáticas como rojo-verde.
- **Impresión en escala de grises:** los colores deben diferenciarse claramente en tonos de gris.

Herramientas como `seaborn.color_palette('colorblind')` y paletas diseñadas con contraste adecuado ayudan a cumplir estos criterios.

***Nota:** Siempre verifica tus gráficos con simuladores de daltonismo y prueba la impresión en escala de grises para asegurar accesibilidad.*

Paletas de color en Matplotlib y Seaborn

Las paletas de color son conjuntos de colores usados para representar datos. Se clasifican principalmente en:

- **Cualitativas:** para variables categóricas sin orden inherente (p.ej., tipos de frutas).
- **Secuenciales:** para variables numéricas ordenadas, donde el color indica magnitud (p.ej., temperatura).
- **Divergentes:** para variables numéricas con un punto medio crítico, mostrando desviaciones en dos direcciones (p.ej., diferencias respecto a un valor base).

Seaborn facilita la selección de paletas con funciones como `color_palette()` y `cubehelix_palette()`, y ofrece paletas predefinidas accesibles y profesionales.

Accesibilidad cromática

Es crucial que las visualizaciones sean interpretables por personas con deficiencias visuales, especialmente daltonismo, y que mantengan su legibilidad al imprimirse en escala de grises.

- **Daltonismo:** afecta aproximadamente al 8% de los hombres y 0.5% de las mujeres. Las paletas deben evitar combinaciones problemáticas como rojo-verde.
- **Impresión en escala de grises:** los colores deben diferenciarse claramente en tonos de gris.

Herramientas como `seaborn.color_palette('colorblind')` y paletas diseñadas con contraste adecuado ayudan a cumplir estos criterios.



Nota: Siempre verifica tus gráficos con simuladores de daltonismo y prueba la impresión en escala de grises para asegurar accesibilidad.

Ciclo de colores y consideraciones prácticas

El ciclo de colores determina cómo se asignan colores a múltiples elementos en un gráfico. Matplotlib permite personalizarlo para mantener coherencia y evitar confusiones.

```
python
from cycler import cyclerplt.rcParams['axes.prop_cycle'] =
cycler(color=['#1f77b4', '#ff7f0e', '#2ca02c'])
```

Esto es especialmente útil en gráficos con múltiples series o categorías.

Resumen de términos clave

Término	Definición breve
rcParams	Diccionario global de configuración en Matplotlib

Término	Definición breve
Paleta cualitativa	Colores para categorías sin orden
Paleta secuencial	Colores para datos numéricos ordenados
Paleta divergente	Colores para datos con punto medio crítico
Daltonismo	Deficiencia visual que afecta percepción de colores
Término	Definición breve
<code>rcParams</code>	Diccionario global de configuración en Matplotlib
Paleta cualitativa	Colores para categorías sin orden
Paleta secuencial	Colores para datos numéricos ordenados
Paleta divergente	Colores para datos con punto medio crítico
Daltonismo	Deficiencia visual que afecta percepción de colores

En la práctica profesional de ciencia de datos, la correcta elección de estilos y paletas de colores impacta directamente en la efectividad de la comunicación visual. Por ejemplo, al presentar un análisis de ventas por región, usar una paleta cualitativa accesible asegura que todos los stakeholders comprendan claramente las diferencias sin confusión visual.

Además, en reportes impresos o presentaciones, garantizar que los gráficos mantengan su legibilidad en escala de grises evita malentendidos y mejora la profesionalidad del trabajo.

Este conocimiento es esencial para diseñar visualizaciones que no solo sean atractivas, sino también inclusivas y éticas, alineándose con las mejores prácticas de la industria y los estándares de accesibilidad.

En la siguiente unidad, aplicaremos estos conceptos para crear composiciones avanzadas y visualizaciones específicas para series temporales y distribuciones, consolidando tu habilidad para diseñar gráficos complejos y efectivos.

6 - Matplotlib: primeros pasos y estructura de figuras

En esta unidad aprenderás a crear figuras y ejes con Matplotlib, trazar líneas y gráficos de dispersión, añadir etiquetas y leyendas, y guardar las figuras con parámetros de resolución y formato. También se introduce la composición de figuras mediante subplots y GridSpec para organizar múltiples gráficos en una sola figura.

Introducción

Matplotlib es una biblioteca fundamental para la visualización de datos en Python. Permite crear gráficos desde simples líneas hasta composiciones complejas con múltiples subplots.

En esta unidad aprenderás a:

- Crear una figura y uno o varios ejes (axes)
- Trazar gráficos de líneas y dispersión
- Añadir etiquetas, títulos y leyendas para mejorar la interpretación
- Guardar las figuras en diferentes formatos y resoluciones
- Organizar múltiples gráficos en una figura usando subplots y GridSpec

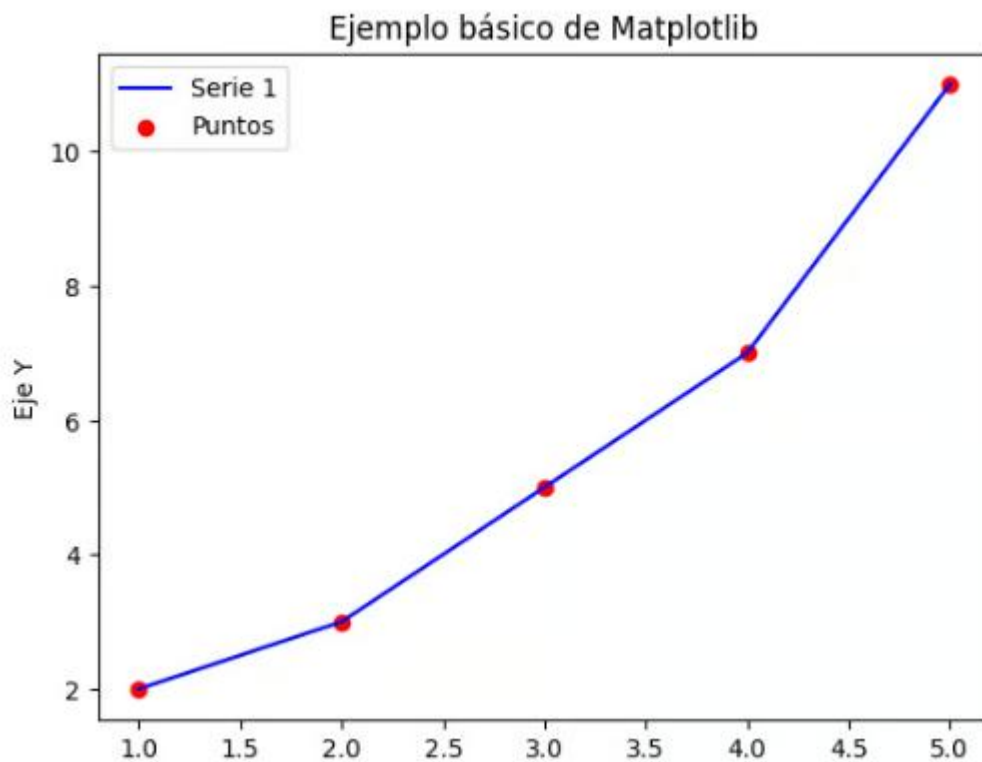
Conceptos clave

- **Figura (figure):** el contenedor principal de uno o varios gráficos.
- **Ejes (axes):** el área donde se dibuja un gráfico.
- `plot`: función para gráficos de líneas.
- `scatter`: función para gráficos de dispersión.
- **Leyendas y etiquetas:** elementos para describir los datos.
- **Guardar figura:** exportar la imagen con parámetros como DPI y formato.
- **Subplots:** múltiples gráficos en una figura.
- **GridSpec:** control avanzado de la disposición de subplots.

Ejemplo básico

```
python
import matplotlib.pyplot as plt
```

```
# Crear figura y ejefig, ax = plt.subplots()
# Datosx = [1, 2, 3, 4, 5]y = [2, 3, 5, 7, 11]
# Gráfico de líneaax.plot(x, y, label='Serie 1', color='blue')
# Gráfico de dispersiónax.scatter(x, y, color='red', label='Puntos')
# Etiquetas y títuloax.set_xlabel('Eje X')ax.set_ylabel('Eje Y')ax.set_title('Ejemplo básico de Matplotlib')
# Leyendaax.legend()
# Guardar figurafig.savefig('grafico.png', dpi=300, format='png')
plt.show()
```



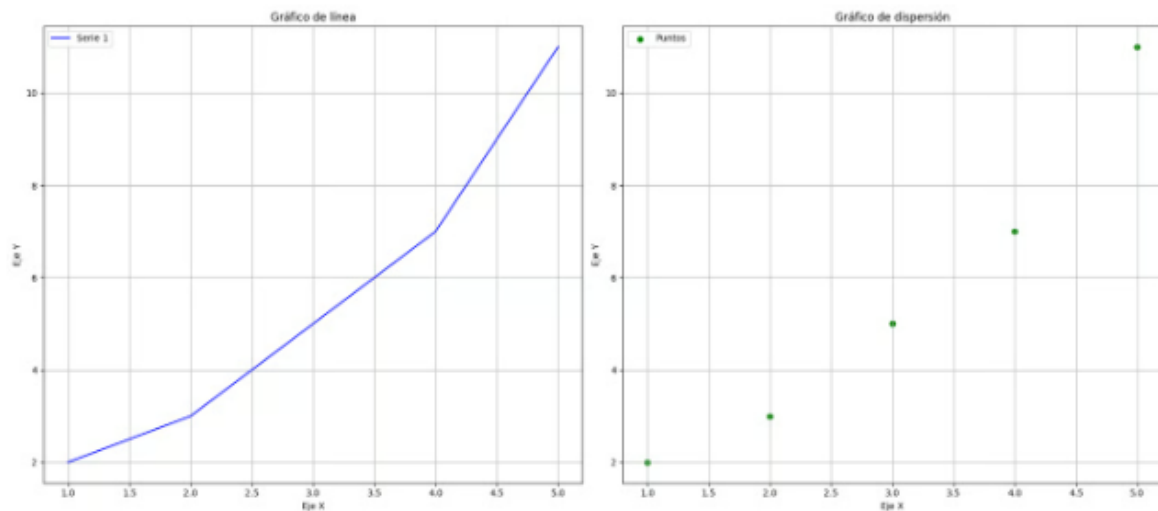
Subplots y GridSpec

Para comparar varias series o vistas, podemos usar subplots:

python

```
import matplotlib.pyplot as plt
# Datosx = [1, 2, 3, 4, 5]y = [2, 3, 5, 7, 11]
# Crear figura con 2 subplotsfig, axes = plt.subplots(1, 2, figsize=(18, 8))
# -----# Primer subplot - Línea# -----
----axes[0].plot(x, y, label='Serie 1', color='blue')axes[0].set_title('Gráfico de
línea')axes[0].set_xlabel('Eje X')axes[0].set_ylabel('Eje
Y')axes[0].legend()axes[0].grid(True)
# -----# Segundo subplot - Dispersión# -----
-----axes[1].scatter(x, y, color='green',
label='Puntos')axes[1].set_title('Gráfico de dispersión')axes[1].set_xlabel('Eje
X')axes[1].set_ylabel('Eje Y')axes[1].legend()axes[1].grid(True)
# Ajustar espaciosplt.tight_layout()
```

```
# Guardar la figurafig.savefig('grafico.png', dpi=300, format='png')
# Mostrarplt.show()
```



GridSpec permite un control más fino de la disposición, pero para esta unidad nos enfocamos en subplots básicos.

GridSpec permite un control más fino de la disposición, pero para esta unidad nos enfocamos en subplots básicos.

7 - Seaborn: visualizaciones relacionales y categóricas

Ejercicio para practicar la creación de visualizaciones relacionales y categóricas usando Seaborn, incluyendo scatterplots, lineplots, boxplots y el uso de FacetGrid para comparar subgrupos.

En esta práctica trabajarás con un conjunto de datos tabulares y deberás generar visualizaciones estadísticas utilizando Seaborn. El objetivo es ejercitar:

-
- Gráficos relacionales (scatter, lineplot)
 - Gráficos categóricos (boxplot)

- Uso de **FacetGrid** para comparar subgrupos
 - Limpieza y adaptación de datos
 - Exportación de gráficos a archivos
-

1 Reproducibilidad y estilo

- **Semilla fija (`_SEED`):**

Establecer una semilla al inicio asegura que cualquier generación aleatoria de datos sea **determinista**, es decir, que al ejecutar varias veces el notebook se obtengan los mismos resultados. Esto es clave en ciencia de datos para depuración y comparación de resultados.

- **Estilo de gráficos (`sns.set_theme`):**

Definir un estilo global con Seaborn hace que todos los gráficos tengan un formato coherente y profesional, evitando diferencias visuales entre celdas.

- **DPI y tamaño de figuras (`mpl.rcParams`):**

Controlar la resolución de los gráficos garantiza que al exportarlos a PNG se mantenga la claridad._

2 Simulación de dataset realista

- **Objetivo:** reproducir un dataset similar al clásico de propinas en restaurantes (`tips`) pero generado de forma aleatoria.
- **Variables a simular:**
 - `total_bill`: monto total de la cuenta (numérico, distribución positiva y sesgada).
 - `tip`: propina, generalmente un porcentaje de `total_bill` más ruido aleatorio.
 - `sex`: género del cliente (Male, Female).
 - `smoker`: si fuma (Yes, No).
 - `day`: día de la semana (Thur, Fri, Sat, Sun).
 - `time`: comida (Lunch, Dinner).
 - `size`: número de personas (Poisson para simular distribución realista).
- **Justificación del método:**
 - `np.random.gamma` para generar montos positivos y sesgados.

- `np.random.normal` para el cálculo de propinas correlacionadas.
 - `np.random.choice` y `np.random.poisson` para variables categóricas y conteos.
 - Ajustes específicos: mayores cuentas en cenas y fines de semana para que la simulación sea más realista.
-

3 Transformación al formato requerido

- **Columnas clave:**
- `group`: subgrupo para usar en `FacetGrid` (p. ej., día del servicio).
- `x`: variable numérica para el eje X (`total_bill`).
- `y`: variable numérica para el eje Y (`tip`).
- `categoria`: variable categórica para color o `boxplot` (`time`).

- **Propósito:**

Transformar el dataset permite **uniformar la estructura de datos** para todas las visualizaciones, facilitando su análisis y comparación. Además, mantener columnas opcionales (`sex`, `smoker`, `size`) permite exploraciones adicionales sin afectar la práctica.

4 Gráficos con Seaborn

a) Scatterplot

- **Qué muestra:** relación entre `total_bill` y `tip`.
- **Color por categoría** (`categoria`): permite distinguir Lunch vs Dinner.
- **Importancia:** visualización relacional básica para detectar patrones o correlaciones.

b) Lineplot (media de y por x)

- **Objetivo:** mostrar la tendencia promedio de `tip` respecto a `total_bill`.
- **Técnica:** agrupar `x` en bins (intervalos), calcular la media de `y` por bin y categoría, y trazar líneas por grupo.
- **Justificación:** ayuda a **resumir relaciones continuas** y suavizar la dispersión de los datos.

c) Boxplot

- **Qué muestra:** distribución de `y` por `categoria`.
- **Importancia:** permite comparar medianas, cuartiles y posibles outliers entre Lunch y Dinner.

- **Ventaja:** identifica diferencias estadísticas entre grupos categóricos.
- d) FacetGrid
- **Qué hace:** crea una **rejilla de subplots** basada en `group` (día), mostrando scatterplots para cada subgrupo.
 - **Uso del color:** mantiene consistencia visual y permite comparar subgrupos fácilmente.
 - **Justificación:** técnica poderosa para analizar patrones por subgrupo sin mezclar datos en un solo gráfico.
-

5 Exportación de gráficos

- Guardar los gráficos como PNG garantiza que los resultados puedan **documentarse y compartirse**, fuera del notebook.
 - Buenas prácticas: usar `plt.savefig` o `FacetGrid.savefig`, incluir `bbox_inches="tight"` para evitar recortes.
-

6 Buenas prácticas de código

- **Funciones claras y testables:** cada paso (simulación, transformación, cada tipo de gráfico) está encapsulado en una función. Esto mejora **modularidad, legibilidad y reutilización**.
 - **Evitar variables globales:** excepto la semilla, todas las operaciones están dentro de funciones o `main()`, lo que asegura **determinismo y reproducibilidad**.
 - **Documentación:** docstrings y comentarios permiten entender rápidamente la intención de cada función y parámetro.
-

7 Flujo de ejecución recomendado

1. Definir semilla y estilo global.
2. Simular dataset con `simulate_tips_dataset()`.
3. Transformar al formato uniforme con `transform_dataset()`.
4. Generar cada gráfico:
 - `plot_scatter()`
 - `plot_line()`

- `plot_box()`
 - `plot_facetgrid()`
5. Guardar todos los gráficos en PNG.
 6. Confirmar que los archivos existan y los gráficos sean correctos.

8 - Distribuciones: histograma, KDE, boxplot y violin

Ejercicio para aplicar visualización de distribuciones usando histogramas, KDE, boxplot y violinplot. Se evalúa la capacidad para interpretar formas de distribución, detectar outliers y elegir el gráfico adecuado según el tipo de datos.

1. Generación de dataset simulado

Para practicar análisis de distribuciones, necesitamos un **dataset realista con valores numéricos** que representen un problema de negocio. Por ejemplo, ventas de un e-commerce:

- La variable `OrderAmount` simula el monto de cada venta.
- La variable puede contener **mezcla de distribuciones** para crear variabilidad:
- **Gamma**: para la mayoría de compras normales.
- **Log-normal**: para compras grandes que generan un “heavy tail”.
- **Uniforme**: para pequeñas promociones o clusters especiales.

Además, agregamos **variables categóricas** (`Category`, `Region`) y fechas (`Date`) para poder hacer análisis más rico en otros contextos.

Justificación: la práctica requiere una distribución **no trivial**, con la posibilidad de detectar **outliers**, **skewness** y **multimodalidad**, simulando situaciones reales.

2. Selección de la serie numérica

Se elige una columna **numérica** para analizar la distribución (`OrderAmount` en este caso).

Justificación: para la visualización de distribuciones necesitamos una **variable continua**, donde tenga sentido calcular histograma, KDE, boxplot o violinplot.

3. Estadísticas descriptivas

Calculamos:

1. **Cuartiles:** Q1 (25%) y Q3 (75%).
2. **Mediana:** 50%.

3. **IQR (Interquartile Range):** $IQR = Q3 - Q1$.

4. **Detección de outliers** usando la regla del boxplot:

$$\begin{aligned}\text{lower bound} &= Q1 - 1.5 \cdot IQR, \text{upper bound} = Q3 + 1.5 \cdot IQR \\ \text{lower bound} &= Q1 - 1.5 \cdot IQR, \text{upper bound} = Q3 + 1.5 \cdot IQR\end{aligned}$$

Cualquier valor fuera de ese rango se considera **outlier**.

Justificación: **

Boxplot y violinplot visualizan dispersión y outliers, y estas métricas son la base de su cálculo.

4. Detección de multimodalidad

- Se calcula un **histograma** de la serie.
- Se suaviza el conteo con una **media móvil** para reducir ruido.
- Se cuentan los **picos locales**: valores mayores que sus vecinos.

Justificación: **

*Detectar si la distribución tiene **más de un pico** ayuda a decidir si conviene un **KDE**, que muestra forma continua y suavizada, frente a un histograma simple.*

5. Cálculo de skewness

- Skewness (asimetría) = tercer momento central normalizado:

$$\begin{aligned}\text{skewness} &= \frac{1}{N} \sum_i (x_i - \bar{x})^3 / \sigma^3 \\ \text{skewness} &= \frac{1}{\sigma^3} \sum_i (x_i - \bar{x})^3\end{aligned}$$

Justificación: **

*Si la distribución es **muy sesgada**, un histograma puede ocultar información; en esos casos, el **violinplot** permite mostrar forma y dispersión de manera más completa.*

6. Lógica de recomendación de gráfico

Se siguen reglas simplificadas basadas en la práctica:

1. **Outliers > 0 → BOXPLOT**

Permite visualizar dispersión y outliers claramente.

2. **Más de un pico → KDE**

Muestra la forma multimodal de la distribución.

3. **Unimodal y sin outliers**

- Si skewness $> 1 \rightarrow$ VIOLIN

Permite apreciar forma y dispersión.

- Si simétrica $\rightarrow$ HISTOGRAMA

Representa la frecuencia de manera simple.

Justificación:**

Esta lógica traduce la **información estadística** en una **decisión visual**, que es el objetivo de la práctica.

7. Impresión de resultados

Se requiere **formato exacto**:

1. Línea 1: nombre del gráfico recomendado (HISTOGRAMA, KDE, BOXPLOT o VIOLIN).
2. Línea 2: cantidad de outliers.
3. Línea 3: mediana e IQR, con 2 decimales.

Justificación:**

Permite validar de manera automatizada y reproducible si la recomendación corresponde a la distribución generada.

8. Visualizaciones opcionales

- **Histograma:** frecuencias por intervalos.
- **KDE:** estimación suavizada de densidad.
- **Boxplot:** mediana, cuartiles y outliers.
- **Violinplot:** combina KDE y boxplot para mostrar forma y dispersión.

Justificación:**

Aunque no es obligatorio, **visualizar todos los gráficos** ayuda a los estudiantes a **confirmar la elección automática**.

9. Información adicional (opcional)

Para fines de depuración o aprendizaje:

- Número de observaciones (N).
- Número de picos detectados.
- Skewness.

- Límites para outliers.

Justificación:**

Permite **relacionar métricas estadísticas** con la decisión del gráfico y comprender cómo afectan la recomendación.

✓ **Resumen conceptual de la práctica:**

- Generar un dataset con **variabilidad realista**.
- Seleccionar una serie numérica y calcular **estadísticas clave** (mediana, IQR, outliers, skewness).
- Detectar **multimodalidad**.
- Tomar decisiones de visualización según reglas simples.
- Visualizar con **histograma, KDE, boxplot y violin** para inspección.
- Reforzar la relación entre **estadística descriptiva y visualización**.

9 - Plotly: crear visualizaciones interactivas

Ejercicio para practicar la creación de gráficos interactivos usando Plotly en Python. Se trabajará con datos de entrada para generar un scatter plot interactivo, con tooltips personalizados y exportación a HTML.

Objetivo: Aprender a construir visualizaciones interactivas con Plotly y Pandas, integrando múltiples dimensiones (x, y, tamaño, color, metadatos) y añadir controles para filtrar la vista.

Descripción:

Se te pedirá que construyas un scatter plot interactivo que muestre puntos (x, y) con una etiqueta por punto. Cada punto debe tener un tooltip enriquecido que muestre la etiqueta y otras características derivadas. Además, debes poder filtrar las etiquetas con un control interactivo (dropdown) y exportar el gráfico final a `grafico_interactivo.html`.

Requisitos (lo que se evaluará):

- Construcción del DataFrame con Pandas (columnas: x, y, label, size, feature1, feature2, etc.).
- Gráfico interactivo Plotly con:
 - Color por label.
 - Tamaño de punto por size.
 - Tooltip que muestre múltiples campos (label, x, y, otras features).
- Control (dropdown) que permita filtrar por etiqueta (ver "All" y etiqueta individual).

10 - Diseño avanzado de subplots con GridSpec en Matplotlib

Implementa un programa que, dado un conjunto de instrucciones para crear una figura compuesta con subplots usando GridSpec, genere la configuración correcta de filas, columnas, tamaños relativos y márgenes. El programa debe procesar la entrada que describe la estructura de la figura y devolver un resumen textual de la composición, validando la correcta asignación de subplots y sus dimensiones relativas.

Objetivo de la práctica

En esta práctica trabajaremos con **diseño avanzado de visualizaciones**, un paso fundamental cuando los análisis de datos deben transformarse en **dashboards profesionales**.

A diferencia de los gráficos individuales, en un entorno real de negocio es habitual presentar:

- múltiples métricas,
- diferentes tipos de gráficos,
- jerarquía visual,
- y una distribución espacial clara.

Para resolver este problema, Matplotlib ofrece la herramienta **GridSpec**, que permite definir layouts complejos de subplots de forma precisa y flexible.

Contexto de negocio

El escenario simula un **e-commerce**, donde el equipo de datos necesita construir un dashboard que permita:

- analizar ventas por categoría,
- observar la evolución temporal,
- comparar segmentos de clientes,
- y evaluar el desempeño regional.

El dataset se genera de forma sintética, pero con **variables realistas**:

- variables numéricas: Price, Quantity, Discount, NetSales
- variables categóricas: Category, Region, CustomerSegment
- variable temporal: OrderDate

El foco de la práctica **no es el análisis exploratorio**, sino el **diseño y validación del layout visual**.

¿Por qué GridSpec?

Cuando usamos:

```
plt.subplot(2, 2, 1)
```

estamos limitados a una grilla rígida.

En cambio, **GridSpec permite**:

- que un subplot ocupe varias filas,
- que otro ocupe varias columnas,
- construir paneles principales y secundarios,
- replicar dashboards reales.

Ejemplo conceptual:

```
+-----+-----+|               || gráfico
grande | gráfico ||               |+-----+-----+-----
----+| gráfico | gráfico ancho |+-----+-----+
```

Este tipo de diseño **no es posible** con subplots tradicionales.

Representación del layout

En esta práctica, el layout se define mediante una estructura explícita:

```
R, C = 3, 4
```

Esto indica que el espacio total se divide en:

- 3 filas
- 4 columnas

Cada subplot se describe con:


```
{  "r": fila_inicial,  "c": columna_inicial,  "h": alto_en_filas,  "w": ancho_en_columnas,  "id": identificador}
```

Por ejemplo:

```
{"r": 0, "c": 2, "h": 2, "w": 2}
```

significa:

- comienza en fila 0, columna 2
- ocupa 2 filas
- ocupa 2 columnas

Este modelo es **equivalente a cómo funciona GridSpec internamente**.

Paso 1 — Validación del layout

Antes de dibujar cualquier gráfico, es fundamental validar el diseño.

¿Por qué?

En aplicaciones reales:

- layouts pueden venir de archivos,
- configuraciones externas,
- o inputs automáticos.

Si el layout es inválido, el dashboard **no debe construirse**.

✓ Validaciones necesarias

1. Tipos correctos

- Todas las posiciones y tamaños deben ser enteros.

2. Valores válidos

- $r, c \geq 0$
- $h, w > 0$

3. Límites del GridSpec

- $r + h \leq R$
- $c + w \leq C$

Esto evita que un subplot se “salga” del grid.

🚫 Detección de solapamientos

El problema más importante es detectar **subplots superpuestos**.

Ejemplo inválido:

- dos subplots ocupan la misma celda del grid.

Estrategia utilizada

Se construye una **grilla auxiliar**:

```
grid = np.zeros((R, C))
```

Luego, para cada subplot:

- se recorren las celdas que ocupa,
- se marca cada posición,
- si una celda ya estaba ocupada → **ERROR**.

Este enfoque es:

- claro,
 - determinístico,
 - fácil de depurar,
 - equivalente a detección de colisiones espaciales.
-

📤 Formato de salida

El ejercicio replica un formato estructurado:

- Si hay solapamiento:

```
ERROR
```

- Si no hay solapamientos:

```
OKid r c h wid r c h w...
```

Esto obliga a:

- respetar formatos exactos,

- separar lógica de validación de visualización,
- pensar como en evaluadores automáticos o pipelines.

📊 Cálculo del tamaño relativo de la figura

Una vez validado el layout, analizamos **cuánto espacio ocupa el dashboard**.

Métrica básica — coverage

Se calcula:

```
coverage = celdas_ocupadas / celdas_totales
```

Donde:

- celdas totales = $R \times C$
- celdas ocupadas = suma de $(h \times w)$ de cada subplot

Esto permite responder preguntas como:

- ¿el layout aprovecha el espacio?
- ¿hay demasiados huecos?
- ¿el diseño es compacto o disperso?

Métrica avanzada — tamaño con márgenes (opcional)

En visualización real existen:

- márgenes,
- espacios entre subplots,
- padding visual.

Por eso se introduce un margen relativo m , que permite estimar el **tamaño físico aproximado** de la figura.

No es un cálculo exacto de píxeles, sino una **métrica conceptual**, útil para:

- comparar diseños,

- evaluar complejidad visual,
- justificar decisiones de layout.

Construcción final del dashboard

Una vez validado todo:

```
gs = GridSpec(R, C)
```

Y cada subplot se crea mediante slicing:

```
ax = fig.add_subplot(gs[r:r+h, c:c+w])
```

Este paso conecta:

- la lógica espacial,
- con la visualización real.

Cada subplot puede contener:

- gráficos de barras,
- series temporales,
- boxplots,
- heatmaps,
- u otros gráficos según el objetivo.

11 - Exportación y ajuste final para reportes y presentaciones

En esta unidad aprenderás a preparar y exportar tus gráficos para su uso en reportes y presentaciones profesionales, dominando formatos, resolución, tipografías y ajustes visuales esenciales para una comunicación clara y efectiva.

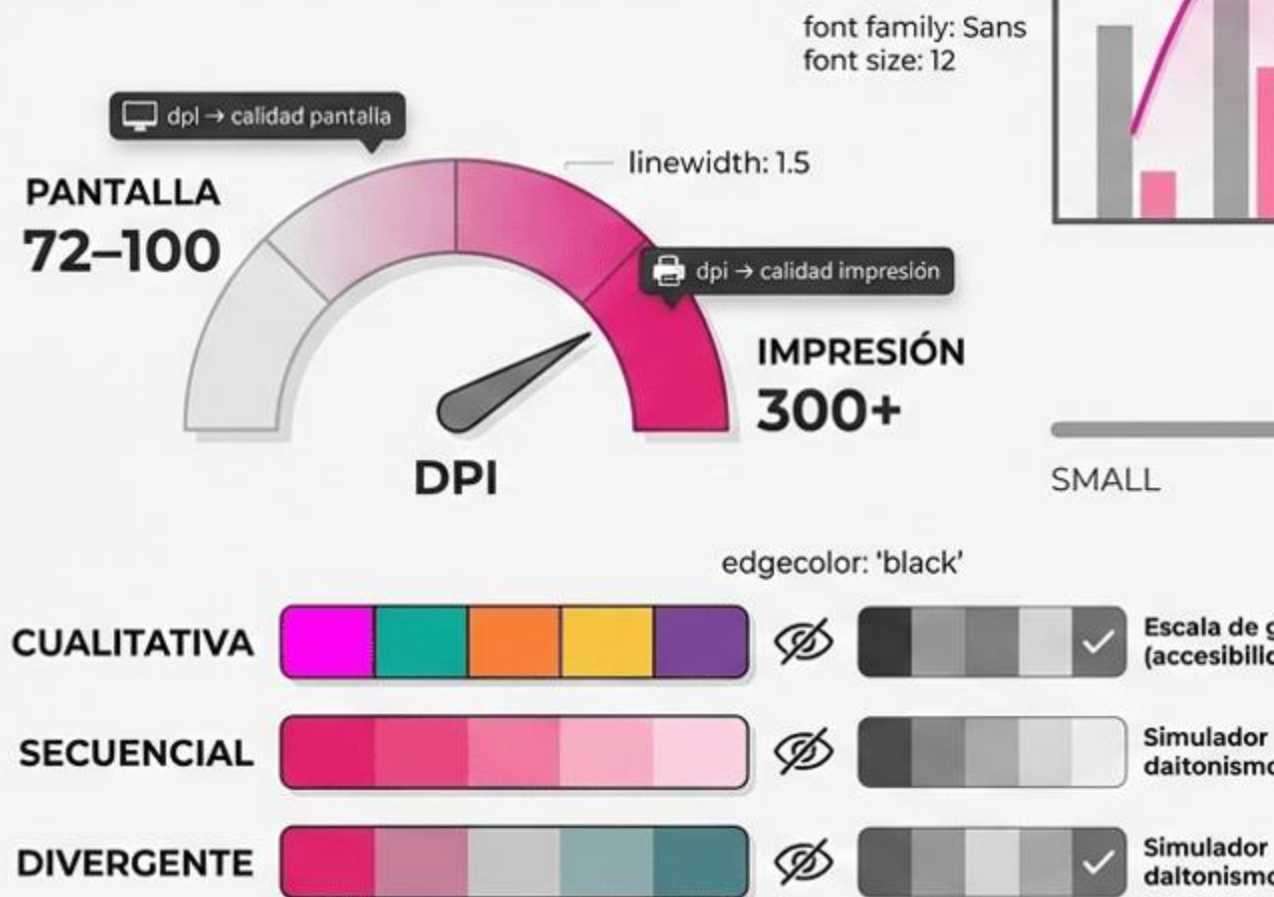
¿Alguna vez has creado un gráfico que se ve perfecto en tu notebook, pero al exportarlo para un reporte o presentación pierde calidad o legibilidad? En esta unidad abordaremos cómo preparar tus visualizaciones para que mantengan su impacto y claridad al ser exportadas y utilizadas en distintos formatos

profesionales. Aprenderás a manejar formatos comunes como PNG, SVG y PDF, ajustar la resolución para impresión o pantalla, y configurar tipografías y estilos para que tus gráficos se integren armónicamente en reportes y diapositivas. Este conocimiento es clave para comunicar insights de forma efectiva y profesional, asegurando que tus visualizaciones reflejen fielmente los datos y el análisis

CODERHOUSE

Exportación y Ajuste Final de Gráficos

formatos, DPI, fontes y export-ready



Exportación y ajuste final de gráficos

Formatos de exportación

- **PNG:** Formato rasterizado, ideal para imágenes en pantalla. Soporta transparencia y buena calidad, pero la resolución depende del DPI configurado.
- **SVG:** Formato vectorial, escalable sin pérdida de calidad. Excelente para gráficos que requieren redimensionarse o editarse posteriormente.
- **PDF:** Formato vectorial para impresión y documentos. Mantiene calidad y es ampliamente compatible con software de edición y presentación.

***Nota:** Elegir el formato adecuado depende del uso final: pantalla (PNG), edición o escalabilidad (SVG), impresión o documentos formales (PDF).*

Resolución y DPI

- **DPI** (dots per inch) determina la calidad de impresión o visualización en pantalla.
- Para presentaciones en pantalla, 72-100 DPI suele ser suficiente.
- Para impresión, se recomienda 300 DPI o más para evitar pixelación.
- En Matplotlib, se configura con el parámetro `dpi` en `savefig()`, por ejemplo:

```
python  
plt.savefig('grafico.png', dpi=300)
```

Tipografías y estilos

- La configuración de estilos en Matplotlib se maneja con `rcParams`, que permite definir tipografías, tamaños, colores y más.
- Seaborn ofrece paletas de color accesibles y configurables para mejorar la legibilidad y estética.
- Es importante mantener la coherencia tipográfica con el resto del reporte o presentación.

Ejemplo de ajuste de tipografía y estilo:

```
python  
import matplotlib.pyplot as plt  
plt.rcParams['font.family'] =  
'Arial'  
plt.rcParams['font.size'] = 12
```

Ajustes para diapositivas e informes

- Tamaño de figura: ajustar con `figsize` para que encaje bien en la diapositiva o página.
- Margenes y etiquetas: asegurarse que no se corten y sean legibles.
- Uso de paletas de color que sean accesibles y contrasten bien.

***Consejo:** Antes de exportar, visualiza el gráfico en el tamaño final para detectar posibles problemas.*

Estilos y paletas (Refuerzo)

- Revisa y ajusta estilos con `plt.style.use()` para aplicar temas profesionales.
- Usa paletas de Seaborn como `deep`, `muted` o `colorblind` para mejorar accesibilidad cromática.

```
python
import seaborn as sns
sns.set_palette('colorblind')
```

Con estos fundamentos, estarás listo para exportar gráficos que comuniquen con claridad y profesionalismo en cualquier contexto.

En la práctica profesional de Ciencia de Datos, la presentación de resultados es tan importante como el análisis mismo. Los gráficos exportados deben conservar su calidad y legibilidad para que los stakeholders puedan interpretar correctamente los insights. Por ejemplo, un gráfico para un reporte impreso requiere alta resolución y tipografías claras, mientras que para una presentación en pantalla se prioriza la visibilidad y el tamaño adecuado. Además, la coherencia visual con el resto del material (colores, fuentes) refuerza la credibilidad y profesionalismo del trabajo.

Este conocimiento es fundamental para entregar proyectos completos y pulidos, facilitando la toma de decisiones basada en datos. En esta unidad, aplicaremos técnicas para ajustar y exportar gráficos usando Matplotlib y Seaborn, herramientas estándar en la industria, asegurando que tus visualizaciones estén listas para cualquier formato final.

12 - Integración en Jupyter y Google Colab; buenas prácticas de reproducibilidad

Ejercicio para aplicar buenas prácticas de reproducibilidad en notebooks: limpieza de celdas, manejo de dependencias y entornos, y documentación inline. Se plantea un problema algorítmico para implementar en un entorno reproducible, enfatizando la organización y claridad del código.

Introducción

En ciencia de datos, escribir código que *funcione* no es suficiente.

Un análisis profesional debe ser **reproducible**, es decir, cualquier persona debe poder ejecutar el notebook desde cero y obtener **los mismos resultados**, sin depender del orden de ejecución, del entorno o de estados previos.

En esta práctica trabajaremos sobre un escenario realista de negocio (e-commerce) y aplicaremos **buenas prácticas de programación en notebooks**, combinando:

- generación controlada de datos,
- funciones bien definidas,
- validación mediante tests,

- y ejecución ordenada del flujo completo.

El objetivo no es solo resolver un problema lógico, sino **aprender a estructurar notebooks como lo haría un/a data scientist en un entorno profesional.**

Objetivos de la práctica

Al finalizar el ejercicio deberías ser capaz de:

- Comprender qué significa reproducibilidad en análisis de datos.
 - Controlar resultados aleatorios mediante semillas.
 - Organizar el código en funciones claras y reutilizables.
 - Implementar un algoritmo simple de forma eficiente.
 - Ejecutar todo el flujo desde una única función principal (`main()`).
 - Validar resultados mediante tests locales.
 - Ejecutar el notebook completo sin errores ni dependencias ocultas.
-

Contexto de negocio

Trabajaremos con un **dataset simulado de e-commerce**, que representa información típica de una tienda online:

- fecha de compra,
- segmento de cliente,
- región,
- categoría de producto,
- precio,
- cantidad,
- descuento,

- ventas netas.

Aunque los datos sean simulados, el objetivo es que **tengan estructura realista**, similar a la que encontrarías en un entorno laboral.

Este dataset será utilizado luego para **aplicar la lógica del problema principal** sobre una columna real del negocio.

🔄 ¿Qué significa que un notebook sea reproducible?

Un notebook reproducible cumple con estas condiciones:

✓ Siempre produce el mismo resultado

Si ejecutamos el notebook hoy o mañana, en otra computadora o en Google Colab, los valores obtenidos deben ser los mismos.

Para lograrlo:

- se fijan semillas aleatorias,
 - no se usan valores generados “al azar” sin control,
 - no se depende del orden de ejecución manual.
-

✓ Puede ejecutarse de arriba hacia abajo

Un notebook reproducible debe funcionar correctamente si alguien presiona:

Runtime → Run all

Esto implica:

- no depender de celdas previas ejecutadas “a mano”,
 - no usar variables definidas fuera del flujo,
 - encapsular la lógica en funciones.
-

✓ No depende del usuario

Por eso, en esta práctica:

- **no se usa** `input()`
- **no se lee desde stdin**
- los datos de ejemplo se construyen directamente en el código

Esto permite que el notebook se ejecute de forma automática, sin intervención humana.

□ Estructura general del ejercicio

La práctica está dividida en bloques claramente definidos.

Cada bloque cumple un propósito específico.

□ Control de reproducibilidad

En este bloque se fijan las semillas aleatorias.

¿Por qué es importante?

Funciones como:

- `np.random`
- `random`
- generación de fechas
- generación de precios o cantidades

producen resultados distintos en cada ejecución si no se controlan.

Al definir una semilla fija:

```
np.random.seed(42)random.seed(42)
```

garantizamos que:

- los mismos números se generen siempre,
- el dataset sea idéntico en cada ejecución,
- los resultados puedan compararse y evaluarse.

Esto es fundamental en notebooks evaluables y en proyectos reales.

🔗 Función central: lógica del problema

El problema algorítmico que se trabaja es:

Calcular la suma de los números que son múltiplos de 3 o de 5.

Este problema se implementa como una **función pura**, lo que significa que:

- recibe datos,
- produce un resultado,
- no modifica variables externas,
- no depende del entorno.

La función debe:

- recorrer la colección una sola vez (eficiencia),
- convertir los valores a enteros,
- verificar la condición matemática,
- acumular el resultado.

Este enfoque es escalable incluso para listas grandes (hasta 100.000 elementos).

🔗 Generación del dataset simulado

En lugar de usar un archivo externo, se construye un dataset desde cero.

¿Por qué?

Porque un notebook reproducible debe ser **autocontenido**.

Dentro de esta función se deben:

- fijar semillas locales,

- generar fechas dentro del último año,
- definir variables categóricas,
- generar precios coherentes según categoría,
- calcular ventas netas a partir de reglas de negocio.

Todo el dataset se construye con pandas y se retorna como un `DataFrame`.

Esto simula una situación real donde no siempre se dispone de datos reales, pero se necesita probar lógica, modelos o reportes.

4 Tests locales

Antes de confiar en un algoritmo, debemos validarlo.

Por eso se incluyen **tests simples**, que permiten verificar que la función:

- funciona con listas vacías,
- funciona cuando no hay múltiplos,
- funciona con valores conocidos,
- produce el resultado esperado para el caso clásico 1..10.

Estos tests cumplen un rol pedagógico clave:

- enseñan a validar código,
- previenen errores silenciosos,
- refuerzan buenas prácticas profesionales.

No se utilizan librerías externas de testing para mantener el notebook simple y autocontenido.

5 Función `main()`

La función `main()` actúa como el **orquestador del notebook**.

Dentro de ella se realiza:

1. impresión de versiones de librerías (control de dependencias),
2. generación del dataset,
3. exploración inicial,
4. resolución del problema con ejemplos,
5. aplicación del algoritmo a datos reales,
6. visualización simple,
7. ejecución de tests.

Este patrón es muy común en proyectos reales y permite que el código:

- sea ordenado,
- sea legible,
- pueda migrarse luego a scripts o pipelines.

6 Ejecución controlada

El bloque final:

```
if __name__ == "__main__":    main()
```

garantiza que:

- el código se ejecute correctamente en notebooks,
- también funcione si se exporta como script,
- no se ejecute automáticamente al importar funciones.

Es una práctica estándar en Python profesional.